

ACTIVIDAD 5

Parte A — Javadoc

- Captura con index.html abierto (se ve tu @author).

PACKAGE **CLASS** USE TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH

Package `es.iescamas.socket`

Class `HiloPorClienteServidor`

`java.lang.Object`
`es.iescamas.socket.HiloPorClienteServidor`

All Implemented Interfaces:
`Runnable`

`public class HiloPorClienteServidor`
`extends Object`
`implements Runnable`

Servidor HTTP concurrente que gestiona peticiones mediante hilos independientes.

`http://localhost:9001/nombre/Ana`

Since:
2026-02-04

Version:
1.0

Author:
Silvia Mª Moreno García

Constructor Summary

Constructors

Constructor	Description
<code>HiloPorClienteServidor(int serverPort)</code>	Constructor del servidor.

Method Summary

All Methods **Instance Methods** **Concrete Methods**

Modifier and Type	Method	Description
<code>void</code>	<code>run()</code>	

- Captura de un método documentado donde se vea @param, @throws y @apiNote.

```
/**
 * Procesa la conexión de un cliente.
 * @param clientSocket Socket del cliente que ha iniciado la conexión.
 * @throws IOException Si ocurre un error en la comunicación por socket.
 * @apiNote Soporta rutas dinámicas como /nombre/Ana y gestiona errores 404.
 */
private void processClientRequest(Socket clientSocket) throws IOException {
    try (clientSocket;
        InputStream in = clientSocket.getInputStream();
        BufferedReader br = new BufferedReader(new InputStreamReader(in, Stand
        OutputStream out = clientSocket.getOutputStream())) {

        // 1) Leer la primera línea: "GET /ruta HTTP/1.1"
        String requestLine = br.readLine();
        if (requestLine == null || requestLine.isBlank()) return;

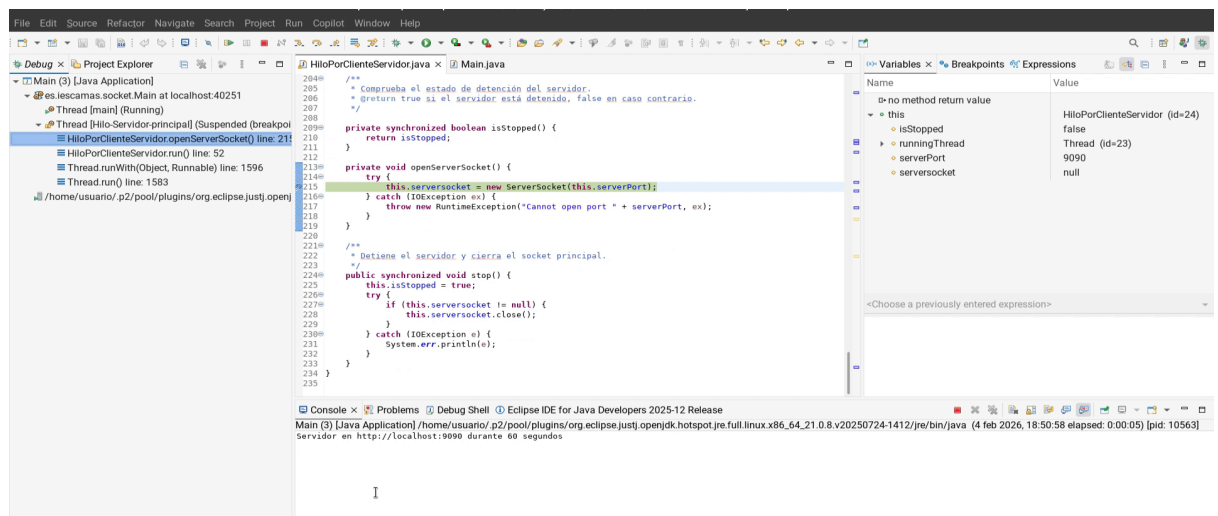
        String path = "/";
```

Parte B — Depuración

Batería de breakpoints (mínimo 7 de 9)

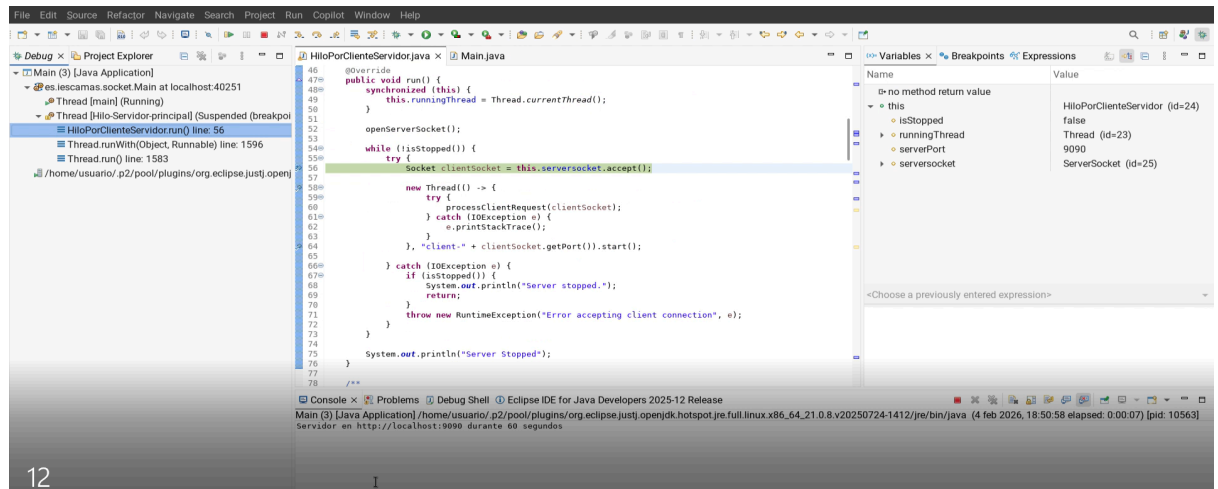
1. B1) Apertura del servidor.

- En openServerSocket() justo después de new ServerSocket(...).
- El servidor intenta reservar el puerto 9090. Está en estado inicial (el objeto serverSocket aún es null antes de ejecutarse la línea). El servidor queda "escuchando" porque el SO abre un canal de entrada en ese puerto específico para recibir peticiones externas.



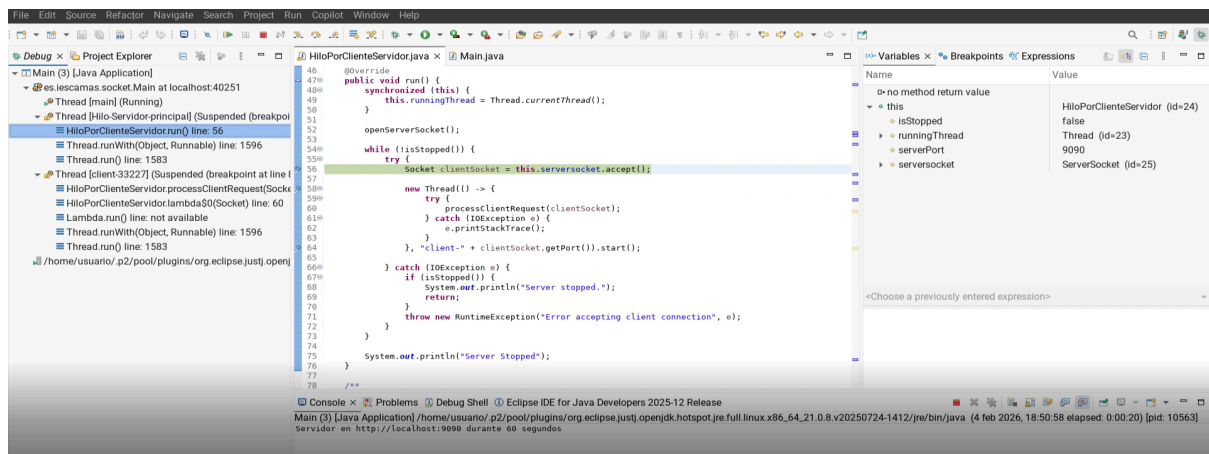
2. B2) Espera de conexiones.

- En `serversocket.accept()`.
- El hilo que se bloquea aquí es el hilo principal (main). Se desbloqueará en el momento exacto en que un cliente realice una petición al puerto 9090.



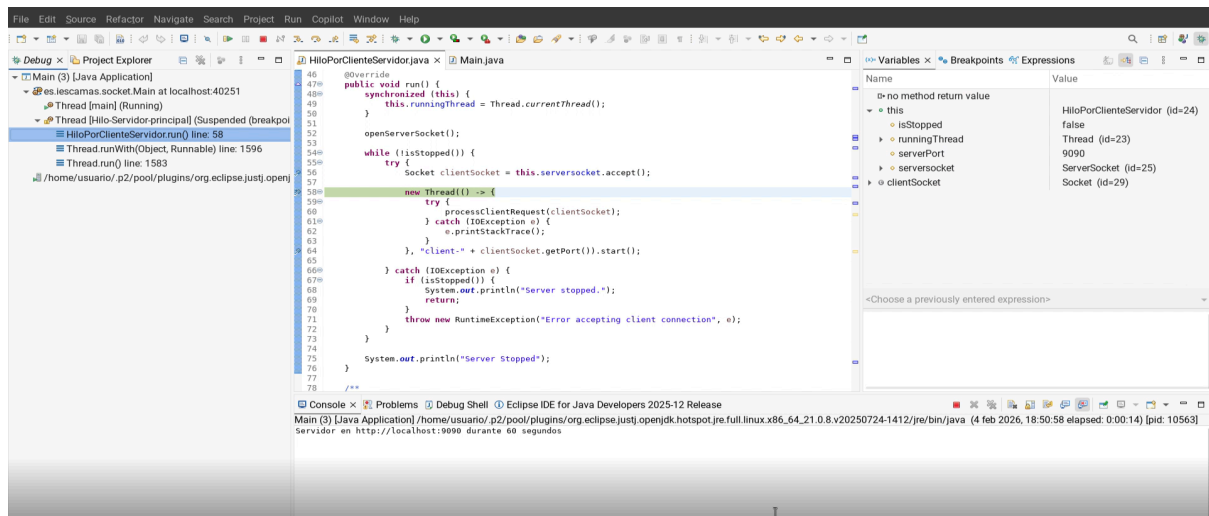
3. B3) Socket aceptado

- Justo después de `Socket clientSocket = ...accept()`.



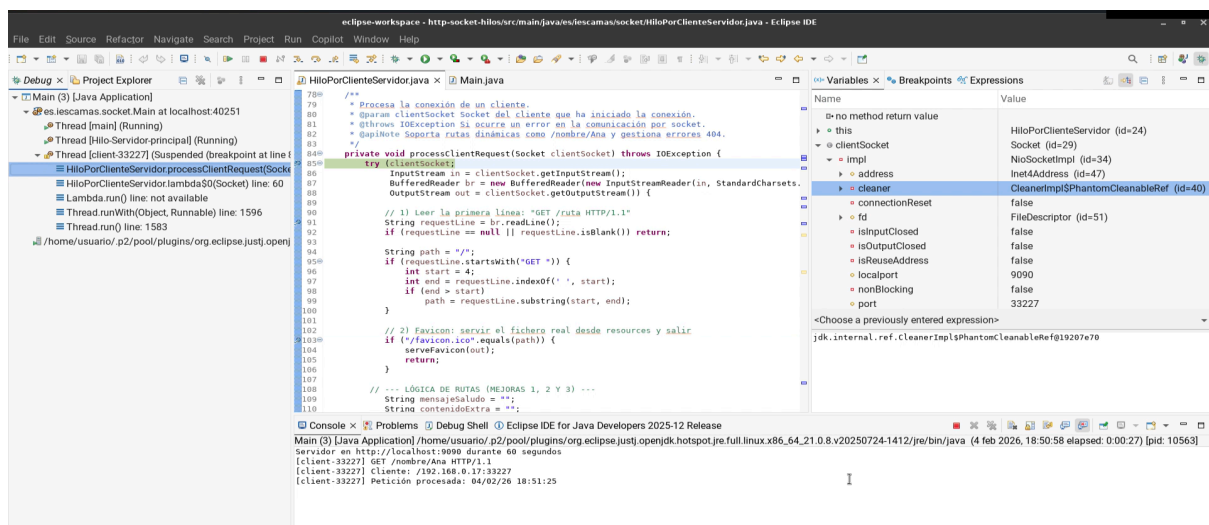
4. B4) Creación del hilo por cliente

- En la línea exacta del `new Thread(...).start()`.
- Se crea un nuevo hilo para no bloquear al servidor y permitir que otros clientes conecten mientras se procesa a Ana.



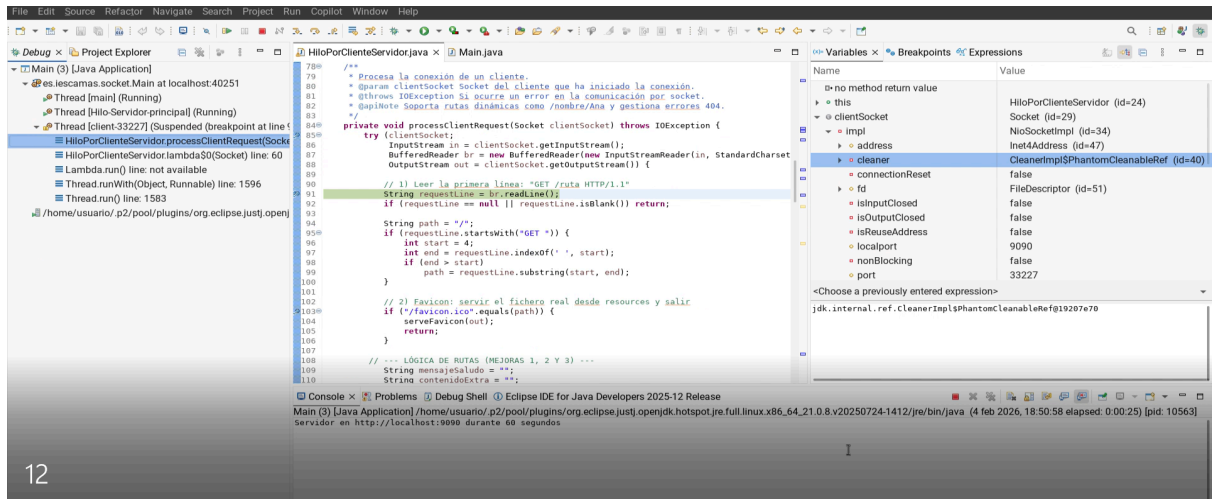
5. B5) Entrada en processClientRequest

- Breakpoint al inicio de processClientRequest(...).



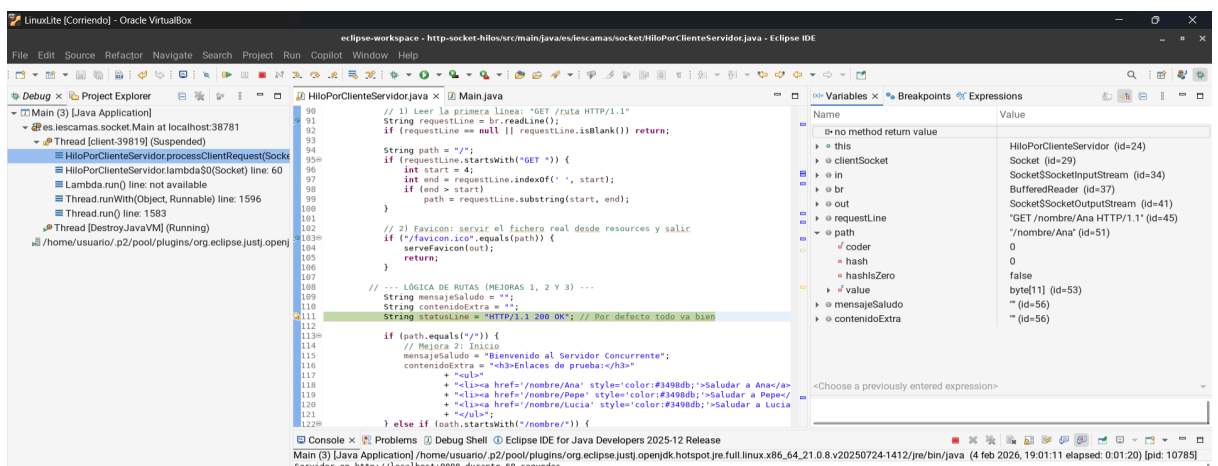
6. B6) Lectura de la primera línea HTTP

- En String requestLine = br.readLine();
- El valor real de requestLine es "GET /nombre/Ana HTTP/1.1". Esto indica que el navegador solicita el recurso /nombre/Ana usando el protocolo HTTP 1.1.

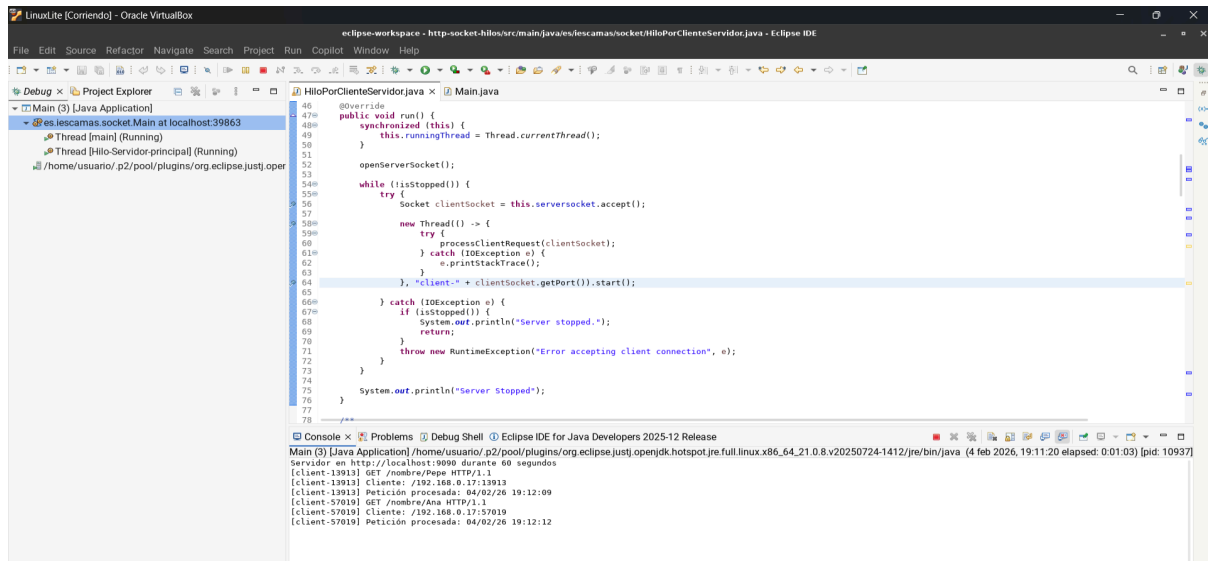


7. B7) Cálculo del path

- Después de calcular path.



Prueba guiada de concurrencia (captura obligatoria)



- En esta captura se demuestra la capacidad multihilo del servidor. Se han lanzado dos peticiones simultáneas desde el navegador. Como se observa en la vista de depuración, existen dos hilos de trabajo (client-13913 y client-57019) detenidos en el breakpoint al mismo tiempo. Cada uno gestiona su propio socket y su propia lógica de saludo sin interferir con el otro, mientras el hilo principal sigue listo para aceptar una tercera conexión.

Parte C — Tests automáticos con JUnit 5

Evidencias Apartado C: Tests Unitarios (JUnit 5)

1. Clase HiloPorClienteServidorTest con los 3 tests y anotaciones.

```
@Test
@DisplayName("C1: GET /nombre/Silvia devuelve 200 OK")
void shouldSayHelloToSilvia() throws Exception {
    // Simula la petición de un cliente real al servidor
    String response = sendRequest("/nombre/Silvia");
    // Verifica (Assert) que la respuesta del servidor contiene el protocolo de éxito
    assertTrue(response.contains("200 OK"), "Debe devolver 200 OK");
    // Comprueba que el saludo dinámico se ha generado correctamente
    assertTrue(response.contains("Hola Silvia"), "Debe saludar a Silvia");
}
```

```

@Test
@DisplayName("C2: Ruta inexistente devuelve 404 Not Found")
void shouldReturn404() throws Exception {
    // Lanza una petición a una ruta que no existe en el servidor
    String response = sendRequest("/ruta-que-no-existe");
    // Valida que el servidor ejecuta la MEJORA 3
    assertTrue(response.contains("404"), "La respuesta debe contener el código de error 404");
}

```

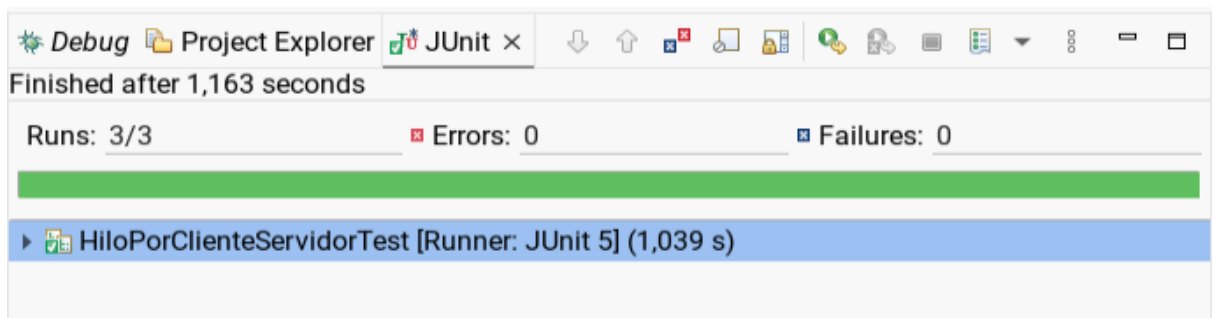
```

@Test
@DisplayName("C3: Concurrencia - Dos clientes a la vez")
@Timeout(value = 5, unit = TimeUnit.SECONDS)
void shouldBeConcurrent() throws Exception {
    // CompletableFuture lanza las peticiones en hilos separados de forma asíncrona
    CompletableFuture<String> res1 = CompletableFuture.supplyAsync(() -> {
        try { return sendRequest("/nombre/Ana"); } catch (Exception e) { return ""; }
    });
    CompletableFuture<String> res2 = CompletableFuture.supplyAsync(() -> {
        try { return sendRequest("/nombre/Pepe"); } catch (Exception e) { return ""; }
    });

    // Verifica que ambos clientes recibieron su respuesta personalizada sin estorbarse
    assertTrue(res1.get().contains("Hola Ana"));
    assertTrue(res2.get().contains("Hola Pepe"));
}

```

2. Captura del resultado en verde en Eclipse o salida de mvn test.



3. 3–6 líneas explicando qué verifica cada test.

- **Test C1 (Saludo)** → Este test comprueba que el servidor "lee" bien lo que escribes en la dirección web. Verifica que si pides /nombre/Silvia, el servidor es capaz de sacar tu nombre de ahí y responderte con un "200 OK" que significa "Todo ha ido bien" y un mensaje que dice "Hola Silvia".
- **Test C2 (Error 404)** → Sirve para ver cómo reacciona el servidor cuando alguien se equivoca al escribir la dirección. Verifica que si pides algo que no existe como /ruta-falsa, el servidor no se rompe, sino que responde con un "404 Not Found". Esto demuestra que la Mejora 3 de gestión de errores funciona correctamente.

- **Test C3 (Concurrencia)** → Este es el test de la "multitarea". Simula que dos personas (Ana y Pepe) entran a la vez en el servidor. El test confirma que el servidor no se queda "atascado" con la primera persona, sino que abre dos hilos de trabajo distintos y les responde a los dos al mismo tiempo.